CSA 4302 - INTERNET PROGRAMMING

LAB EXPERIMENT 04:

AIM : To design and develop an Online Train Reservation System that enables passengers to register, log in, search for trains, check seat availability, select travel classes, and book tickets for specific journeys. The system generates tickets and PNR numbers after successful payment, allows passengers to check PNR status, view booking history, and cancel tickets. It also provides an administrator module to manage train schedules, fares, seat availability, payment details, cancellations, and refunds, ensuring efficient and convenient railway reservation management.

CODE:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Cornerstone Library</title>
  <style>
    :root {
      color-scheme: light;
      --ink: #24312d;
      --muted: #68756f;
      --paper: #f7f8f4;
      --card: #fff;
      --green: #27634f;
      --green-dark: #194b3a;
      --line: #e3e9e3;
      --accent: #eaf2ed;
      --danger: #a43c34;
      font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont,
"Segoe UI", sans-serif;
    }
    * { box-sizing: border-box; }
    body { margin: 0; background: var(--paper); color: var(--ink); }
    button, input, select, textarea { font: inherit; }
    button { cursor: pointer; }
    header { background: var(--card); border-bottom: 1px solid var(--line); }
    .header-inner { max-width: 1100px; margin: auto; padding: 16px 24px; display:
flex; align-items: center; justify-content: space-between; gap: 18px; }
    .brand { font-size: 1.15rem; font-weight: 750; color: var(--green-dark); letter-
spacing: -.03em; }
    .brand span { color: #9a7952; }
    nav { display: flex; gap: 7px; flex-wrap: wrap; }
```

```
    nav button, .link-button { background: transparent; border: 0; color:
var(--muted); padding: 9px 11px; border-radius: 8px; }
    nav button:hover, nav button.active { background: var(--accent); color: var(--green-dark); }
    .header-actions { display: flex; align-items: center; gap: 10px; }
    .user-label { color: var(--muted); font-size: .9rem; }
    .container { max-width: 1100px; padding: 36px 24px 60px; margin: auto; }
    .hero { display: grid; grid-template-columns: 1.4fr .8fr; gap: 28px; align-items:
center; background: #eaf1e9; border-radius: 18px; padding: 34px; margin-bottom: 30px;
}
    .eyebrow { color: var(--green); text-transform: uppercase; letter-spacing: .12em;
font-size: .75rem; font-weight: 700; }
    h1 { font-size: clamp(2rem, 5vw, 3.2rem); line-height: 1.05; letter-spacing: -
.05em; margin: 10px 0 14px; }
    h2 { letter-spacing: -.03em; margin: 0 0 8px; }
    h3 { margin: 0 0 8px; }
    p { color: var(--muted); line-height: 1.6; }
    .hero p { max-width: 530px; margin: 0 0 20px; }
    .hero-art { min-height: 185px; border-radius: 14px; background:
linear-gradient(145deg, #cbdccd, #e8d9bd); display: grid; place-items: center; font-
size: 5rem; }
    .button { border: 1px solid var(--green); border-radius: 8px; padding: 9px 13px;
background: var(--green); color: white; font-weight: 650; }
    .button:hover { background: var(--green-dark); }
    .button.secondary { color: var(--green-dark); background: white; }
    .button.secondary:hover { background: var(--accent); }
    .button.danger { background: white; color: var(--danger); border-color: #e5b9b5; }
    .button.small { padding: 6px 9px; font-size: .86rem; }
    .section-heading { display: flex; justify-content: space-between; align-items:
end; gap: 14px; margin: 28px 0 15px; }
    .section-heading p { margin: 4px 0 0; }
    .grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
    .card { background: var(--card); border: 1px solid var(--line); border-radius:
12px; padding: 18px; }
    .book-card { display: flex; flex-direction: column; min-height: 195px; }
    .book-card .meta { color: var(--muted); font-size: .9rem; }
    .book-card .availability { margin-top: auto; padding-top: 14px; display: flex;
justify-content: space-between; align-items: center; gap: 12px; }
    .badge { border-radius: 99px; background: var(--accent); color: var(--green-dark);
padding: 4px 9px; font-size: .78rem; font-weight: 650; }
    .badge.unavailable { color: var(--danger); background: #f9e9e7; }
    .toolbar { display: flex; gap: 10px; margin-bottom: 16px; }
    input, select, textarea { width: 100%; padding: 10px 11px; border-radius: 8px;
border: 1px solid #d7dfd8; background: white; color: var(--ink); }
    input:focus, select:focus, textarea:focus { outline: 2px solid #bad5c6; border-
color: var(--green); }
    label { display: grid; gap: 6px; color: #43514b; font-size: .9rem; font-weight:
600; }
    form { display: grid; gap: 12px; }
```

```
    .form-grid { display: grid; grid-template-columns: repeat(2, 1fr); gap: 12px; }
    .stack { display: grid; gap: 16px; }
    .table-wrap { overflow-x: auto; }
    table { border-collapse: collapse; width: 100%; text-align: left; font-size:
.91rem; }
    th, td { padding: 11px 10px; border-bottom: 1px solid var(--line); vertical-align:
middle; }
    th { color: var(--muted); font-weight: 650; font-size: .82rem; }
    .empty { padding: 26px; text-align: center; color: var(--muted); background:
white; border: 1px dashed #cdd8cf; border-radius: 10px; }
    .notice { padding: 12px 14px; border-radius: 8px; background: var(--accent);
color: var(--green-dark); margin-bottom: 14px; }
    .notice.error { background: #f9e9e7; color: var(--danger); }
    .fine { font-weight: 700; color: var(--danger); }
    .hidden { display: none !important; }
    .modal { position: fixed; inset: 0; z-index: 3; background: #15231dcc; display:
grid; place-items: center; padding: 18px; }
    .modal-card { width: min(440px, 100%); background: white; border-radius: 14px;
padding: 24px; box-shadow: 0 16px 60px #0003; }
    .modal-head { display: flex; justify-content: space-between; align-items: center;
margin-bottom: 16px; }
    .close { border: 0; background: transparent; font-size: 1.4rem; color:
var(--muted); }
    footer { border-top: 1px solid var(--line); padding: 23px; text-align: center;
color: var(--muted); font-size: .88rem; }
    @media (max-width: 760px) {
      .header-inner { align-items: flex-start; flex-wrap: wrap; }
      .header-actions { margin-left: auto; }
      nav { order: 3; width: 100%; }
      .container { padding: 25px 16px 45px; }
      .hero { grid-template-columns: 1fr; padding: 25px; }
      .hero-art { min-height: 130px; }
      .grid { grid-template-columns: repeat(2, 1fr); }
    }
    @media (max-width: 520px) {
      .grid, .form-grid { grid-template-columns: 1fr; }
      .toolbar { flex-direction: column; }
      .header-inner { padding: 13px 16px; }
      nav { overflow-x: auto; flex-wrap: nowrap; }
      nav button { white-space: nowrap; }
      .user-label { display: none; }
    }
  </style>
</head>
<body>
  <header>
    <div class="header-inner">
      <div class="brand">Cornerstone <span>Library</span></div>
```

```
        <nav aria-label="Main navigation">
          <button class="active" data-page="home">Home</button>
          <button data-page="catalog">Book catalog</button>
          <button data-page="materials">Study materials</button>
          <button data-page="dashboard" class="hidden" id="dashboard-nav">My
account</button>
          <button data-page="admin" class="hidden" id="admin-nav">Librarian</button>
          <button data-page="contact">Contact</button>
        </nav>
        <div class="header-actions">
          <span class="user-label" id="user-label"></span>
          <button class="button secondary small" id="login-open">Log in</button>
          <button class="button small hidden" id="logout">Log out</button>
        </div>
      </div>
    </header>
    <main class="container">
      <div id="message" class="notice hidden" role="status"></div>
      <section id="page-home" class="page">
        <div class="hero">
          <div>
            <div class="eyebrow">Your campus library</div>
            <h1>Find your next<br>great read.</h1>
            <p>Search the library catalog, explore study materials, and keep track of
the books you borrow—all in one place.</p>
            <button class="button" data-go="catalog">Explore the catalog</button>
          </div>
          <div class="hero-art" aria-hidden="true">📚</div>
        </div>
        <div class="section-heading">
          <div><h2>Popular books</h2><p>A few picks from our library shelves.</p></div>
          <button class="link-button" data-go="catalog">View all books →</button>
        </div>
        <div id="featured-books" class="grid"></div>
      </section>
      <section id="page-catalog" class="page hidden">
        <div class="section-heading"><div><div class="eyebrow">Browse</div><h2>Book
catalog</h2><p>Search by title, author, or subject.</p></div></div>
        <div class="toolbar"><input id="book-search" type="search" placeholder="Search
books…" aria-label="Search books"></div>
        <div id="book-results" class="grid"></div>
      </section>
      <section id="page-materials" class="page hidden">
        <div class="section-heading"><div><div class="eyebrow">For your
studies</div><h2>Study materials</h2><p>Find resources by academic year and
semester.</p></div></div>
        <div class="toolbar">
```

```
        <select id="material-year" aria-label="Filter by year"><option value="">All years</option></select>
        <select id="material-semester" aria-label="Filter by semester"><option value="">All semesters</option></select>
      </div>
      <div id="material-results" class="grid"></div>
    </section>
    <section id="page-dashboard" class="page hidden">
      <div class="section-heading"><div><div class="eyebrow">Your account</div><h2>My library</h2><p id="account-greeting"></p></div></div>
      <div class="card" style="margin-bottom:16px"><h3>My profile</h3><div id="profile-details"></div></div>
      <div class="card"><h3>Borrow history</h3><p>Current overdue fines are calculated at $1 per day.</p><div id="history-table" class="table-wrap"></div></div>
    </section>
    <section id="page-admin" class="page hidden">
      <div class="section-heading"><div><div class="eyebrow">Library management</div><h2>Librarian dashboard</h2><p>Manage books, loans, and study materials.</p></div></div>
      <div class="stack">
        <div class="card"><h3>Add a book</h3>
          <form id="book-form">
            <input type="hidden" name="bookId">
            <div class="form-grid"><label>Book title<input name="title" required></label><label>Author<input name="author" required></label><label>Subject<input name="subject" required></label><label>Copies<input name="copies" type="number" min="1" value="1" required></label></div>
            <div><button class="button" type="submit">Add book</button> <button class="button secondary hidden" id="cancel-book-edit" type="button">Cancel edit</button></div>
          </form>
        </div>
        <div class="card"><h3>Inventory</h3><div id="inventory-table" class="table-wrap"></div></div>
        <div class="card"><h3>Issue a book</h3>
          <form id="issue-form">
            <div class="form-grid"><label>Student account<select name="user" id="student-select" required></select></label><label>Available book<select name="book" id="issue-book-select" required></select></label><label>Expected return date<input name="due" type="date" required></label></div>
            <button class="button" type="submit">Issue book</button>
          </form>
        </div>
        <div class="card"><h3>Issued books &amp;amp; returns</h3><div id="loans-table" class="table-wrap"></div></div>
        <div class="card"><h3>Add study material</h3>
          <form id="material-form">
```

```
            <input type="hidden" name="materialId">
            <div class="form-grid"><label>Material title<input name="title"
required></label><label>Year<input name="year" type="number" min="1" max="8"
placeholder="e.g. 1" required></label><label>Semester<input name="semester"
type="number" min="1" max="12" placeholder="e.g. 1" required></label><label>File<input
name="file" type="file" accept=".pdf,.doc,.docx,.ppt,.pptx,.txt"></label></div>
            <div><button class="button" type="submit">Add material</button> <button
class="button secondary hidden" id="cancel-material-edit" type="button">Cancel
edit</button></div>
          </form>
          <div id="admin-materials" class="table-wrap" style="margin-top:18px"></div>
        </div>
      </div>
    </section>
    <section id="page-contact" class="page hidden">
      <div class="section-heading"><div><div class="eyebrow">Get in
touch</div><h2>Contact the library</h2><p>Send the librarian a
message.</p></div></div>
      <div class="card" style="max-width:620px">
        <form id="contact-form">
          <label>Your name<input name="name" required></label><label>Email<input
type="email" name="email" required></label><label>Message<textarea name="message"
rows="5" required></textarea></label>
          <button class="button" type="submit">Send message</button>
        </form>
      </div>
    </section>
  </main>
  <footer>Cornerstone Library · Open for your next chapter</footer>
  <div id="auth-modal" class="modal hidden" role="dialog" aria-modal="true" aria-
labelledby="auth-title">
    <div class="modal-card">
      <div class="modal-head"><h2 id="auth-title">Log in</h2><button class="close"
id="modal-close" aria-label="Close">×</button></div>
      <div class="toolbar"><button class="button small" id="show-login">Log
in</button><button class="button secondary small" id="show-register">Create
account</button></div>
      <form id="login-form"><label>Email<input name="email" type="email" required
autocomplete="username"></label><label>Password<input name="password" type="password"
required autocomplete="current-password"></label><button class="button"
type="submit">Log in</button></form>
      <form id="register-form" class="hidden"><label>Name<input name="name" required
autocomplete="name"></label><label>Email<input name="email" type="email" required
autocomplete="email"></label><label>Password<input name="password" type="password"
minlength="4" required autocomplete="new-password"></label><button class="button"
type="submit">Create account</button></form>
      <p style="font-size:.82rem;margin-bottom:0">Demo librarian:
librarian@library.test / library123</p>
```

```
    </div>
  </div>
  <script>
    const KEY = "cornerstone-library-v1";
    const initial = {
      users: [
        { name: "Library Admin", email: "librarian@library.test", password:
"library123", role: "admin" }
      ],
      books: [
        { id: 1, title: "The Great Gatsby", author: "F. Scott Fitzgerald", subject:
"Literature", copies: 3 },
        { id: 2, title: "A Brief History of Time", author: "Stephen Hawking", subject:
"Science", copies: 2 },
        { id: 3, title: "Introduction to Algorithms", author: "Thomas H. Cormen",
subject: "Computer Science", copies: 4 },
        { id: 4, title: "Pride and Prejudice", author: "Jane Austen", subject:
"Literature", copies: 2 },
        { id: 5, title: "The Selfish Gene", author: "Richard Dawkins", subject:
"Biology", copies: 1 },
        { id: 6, title: "The Art of Statistics", author: "David Spiegelhalter",
subject: "Mathematics", copies: 2 }
      ],
      loans: [],
      materials: [
        { id: 1, title: "Introduction to Biology – Notes", year: "1", semester: "1",
filename: "biology-notes.txt", data: "Welcome to the library study materials. Replace
this sample with your course notes." },
        { id: 2, title: "Calculus – Revision Guide", year: "1", semester: "2",
filename: "calculus-guide.txt", data: "Sample revision guide. Librarians can add real
course files from the dashboard." },
        { id: 3, title: "Database Systems – Lecture Notes", year: "2", semester: "1",
filename: "database-notes.txt", data: "Sample lecture notes. Upload a PDF or document
in the librarian dashboard." }
      ]
    };
    let db;
    try {
      db = JSON.parse(localStorage.getItem(KEY)) || initial;
      if (!Array.isArray(db.users) || !Array.isArray(db.books) ||
!Array.isArray(db.loans) || !Array.isArray(db.materials)) throw new Error("Invalid
saved library data");
    } catch (error) {
      console.error("Could not load saved library data:", error);
      db = JSON.parse(JSON.stringify(initial));
    }
    let currentUser = null;
    const $ = (selector) => document.querySelector(selector);
```

```
    const escapeHTML = (value) => String(value).replace(/[&<>"']/g, c => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" })[c]);
    function save() { localStorage.setItem(KEY, JSON.stringify(db)); }
    function notice(text, error = false) {
      const el = $("#message"); el.textContent = text; el.classList.remove("hidden", "error");
      if (error) el.classList.add("error");
      window.scrollTo({ top: 0, behavior: "smooth" });
      setTimeout(() => el.classList.add("hidden"), 4500);
    }
    function available(book) {
      return book.copies - db.loans.filter(loan => loan.bookId === book.id && !loan.returned).length;
    }
    function fine(loan) {
      if (loan.returned) return 0;
      const days = Math.floor((Date.now() - new Date(loan.due + "T23:59:59").getTime()) / 86400000);
      return Math.max(0, days) * 1;
    }
    function go(page) {
      document.querySelectorAll(".page").forEach(section => section.classList.toggle("hidden", section.id !== `page-${page}`));
      document.querySelectorAll("nav [data-page]").forEach(button => button.classList.toggle("active", button.dataset.page === page));
      if (page === "catalog") renderBooks();
      if (page === "materials") renderMaterials();
      if (page === "dashboard") renderDashboard();
      if (page === "admin") renderAdmin();
      window.scrollTo(0, 0);
    }
    function bookCard(book) {
      const count = available(book);
      return `<article class="card book-card"><h3>${escapeHTML(book.title)}</h3><div class="meta">by ${escapeHTML(book.author)}</div><div class="meta">${escapeHTML(book.subject)}</div><div class="availability"><span class="badge ${count < 1 ? "unavailable" : ""}">${count} available</span><button class="button secondary small details-button" data-id="${book.id}">Details</button></div></article>`;
    }
    function renderBooks() {
      const query = ($("#book-search").value || "").trim().toLowerCase();
      const books = db.books.filter(book => `${book.title} ${book.author} ${book.subject}`.toLowerCase().includes(query));
      $("#book-results").innerHTML = books.length ? books.map(bookCard).join("") : '<div class="empty">No books found. Try another search.</div>';
      document.querySelectorAll(".details-button").forEach(button => button.addEventListener("click", () => {
```

```
        const book = db.books.find(item => item.id === Number(button.dataset.id));
        if (book) notice(`${book.title} by ${book.author} · ${book.subject} · ${available(book)} copies available.`);
      }));
    }
    function renderMaterials() {
      const years = [...new Set(db.materials.map(item => item.year))].sort((a,b) => Number(a)-Number(b));
      const semesters = [...new Set(db.materials.map(item => item.semester))].sort((a,b) => Number(a)-Number(b));
      const year = $("#material-year").value, semester = $("#material-semester").value;
      $("#material-year").innerHTML = '<option value="">All years</option>' + years.map(value => `<option value="${escapeHTML(value)}">${escapeHTML(value)}</option>`).join("");
      $("#material-semester").innerHTML = '<option value="">All semesters</option>' + semesters.map(value => `<option value="${escapeHTML(value)}">${escapeHTML(value)}</option>`).join("");
      $("#material-year").value = year; $("#material-semester").value = semester;
      const filtered = db.materials.filter(item => (!year || item.year === year) && (!semester || item.semester === semester));
      $("#material-results").innerHTML = filtered.length ? filtered.map(item => `<article class="card"><div class="eyebrow">Year ${escapeHTML(item.year)} · Semester ${escapeHTML(item.semester)}</div><h3 style="margin-top:10px">${escapeHTML(item.title)}</h3><a class="button secondary small" style="display:inline-block;text-decoration:none;margin-top:10px" href="${item.data || ""}" download="${escapeHTML(item.filename)}">Download material</a></article>`).join("") : '<div class="empty">No materials match these filters.</div>';
    }
    function renderDashboard() {
      if (!currentUser || currentUser.role !== "user") return go("home");
      $("#account-greeting").textContent = `Welcome back, ${currentUser.name}.`;
      $("#profile-details").innerHTML = `<p><strong>Name:</strong> ${escapeHTML(currentUser.name)}<br><strong>Email:</strong> ${escapeHTML(currentUser.email)}</p>`;
      const loans = db.loans.filter(loan => loan.email === currentUser.email).sort((a,b) => b.issue.localeCompare(a.issue));
      $("#history-table").innerHTML = loans.length ? `<table><thead><tr><th>Book</th><th>Issue date</th><th>Due date</th><th>Status</th><th>Fine</th></tr></thead><tbody>${loans.map(loan => `<tr><td>${escapeHTML(db.books.find(book => book.id === loan.bookId)?.title || "Book removed")}</td><td>${escapeHTML(loan.issue)}</td><td>${escapeHTML(loan.due)}</td><td>${loan.returned ? "Returned" : (fine(loan) ? "Overdue" : "Issued")}</td><td class="${fine(loan) ? "fine" : ""}">$${fine(loan)}</td></tr>`).join("")}</tbody></table>` : '<div class="empty">No borrowed books yet.</div>';
    }
```

```
    function renderAdmin() {
      if (!currentUser || currentUser.role !== "admin") return go("home");
      $("#inventory-table").innerHTML =
`<table><thead><tr><th>Book</th><th>Author</th><th>Copies</th><th>Available</th><th>Ac
tion</th></tr></thead><tbody>${db.books.map(book =>
`<tr><td>${escapeHTML(book.title)}</td><td>${escapeHTML(book.author)}</td><td>${book.c
opies}</td><td>${available(book)}</td><td><button class="button secondary small edit-
book" data-id="${book.id}">Edit</button> <button class="button danger small delete-
book" data-id="${book.id}">Delete</button></td></tr>`).join("") || '<tr><td
colspan="5">No books in inventory.</td></tr>'}</tbody></table>`;
      document.querySelectorAll(".edit-book").forEach(button =>
button.addEventListener("click", () => {
        const book = db.books.find(item => item.id === Number(button.dataset.id));
        if (!book) return;
        const form = $("#book-form");
        form.elements.bookId.value = book.id;
        form.elements.title.value = book.title;
        form.elements.author.value = book.author;
        form.elements.subject.value = book.subject;
        form.elements.copies.min = book.copies - available(book);
        form.elements.copies.value = book.copies;
        form.querySelector('[type="submit"]').textContent = "Update book";
        $("#cancel-book-edit").classList.remove("hidden");
        form.scrollIntoView({ behavior: "smooth", block: "center" });
      }));
      document.querySelectorAll(".delete-book").forEach(button =>
button.addEventListener("click", () => {
        const id = Number(button.dataset.id);
        if (db.loans.some(loan => loan.bookId === id && !loan.returned)) return
notice("Return all copies before deleting this book.", true);
        db.books = db.books.filter(book => book.id !== id); save(); renderAdmin();
renderBooks();
      }));
      const students = db.users.filter(user => user.role === "user");
      $("#student-select").innerHTML = students.map(user => `<option
value="${escapeHTML(user.email)}">${escapeHTML(user.name)}
(${escapeHTML(user.email)})</option>`).join("");
      $("#issue-book-select").innerHTML = db.books.filter(book => available(book) >
0).map(book => `<option value="${book.id}">${escapeHTML(book.title)}
(${available(book)} available)</option>`).join("");
      $("#issue-form button[type=submit]").disabled = !students.length ||
!db.books.some(book => available(book) > 0);
      const loans = [...db.loans].sort((a,b) => b.issue.localeCompare(a.issue));
      $("#loans-table").innerHTML = loans.length ?
`<table><thead><tr><th>Student</th><th>Book</th><th>Issued</th><th>Due</th><th>Fine</t
h><th>Status</th></tr></thead><tbody>${loans.map(loan =>
`<tr><td>${escapeHTML(loan.email)}</td><td>${escapeHTML(db.books.find(book => book.id
=== loan.bookId)?.title || "Book
```

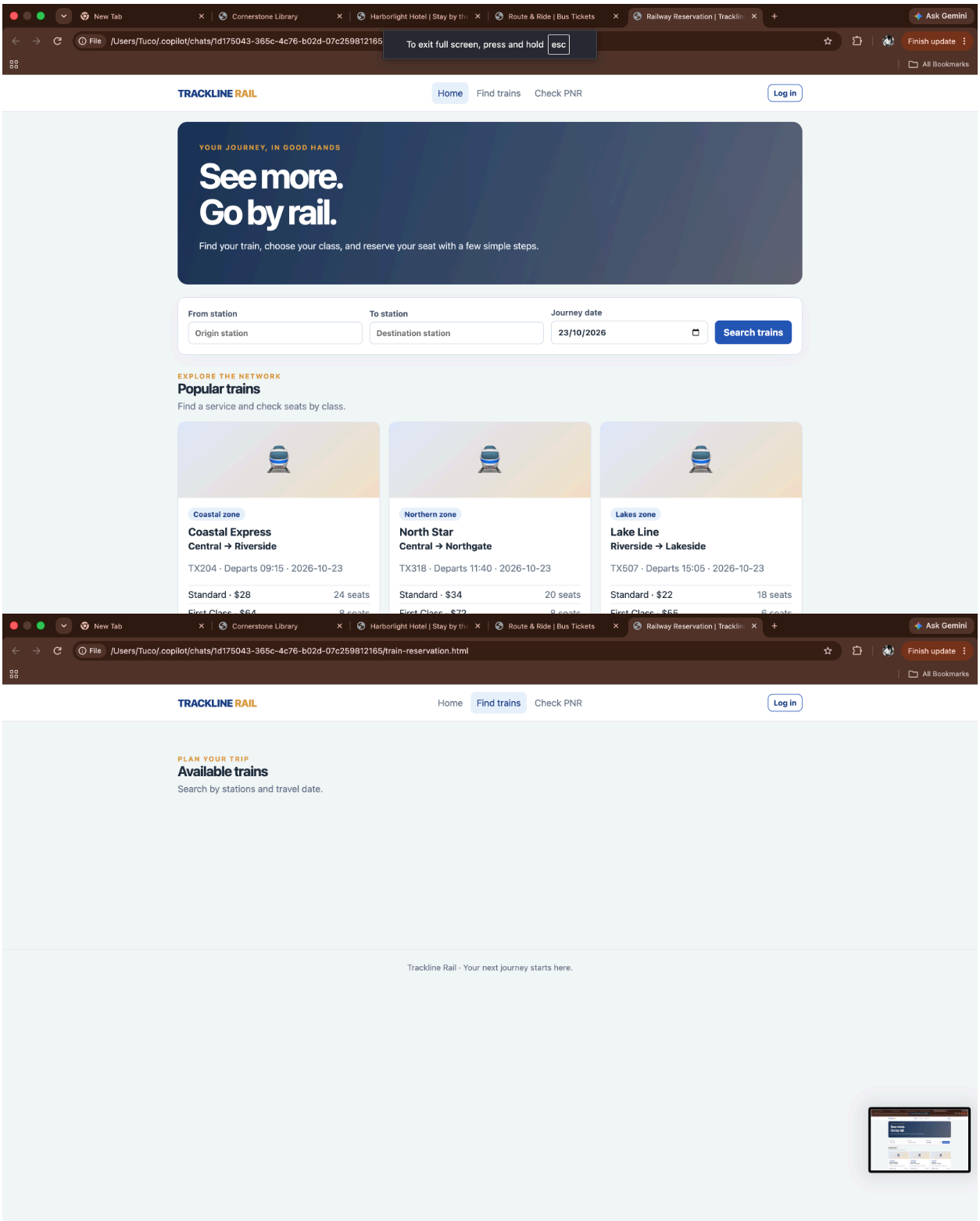
```
removed")}</td><td>${escapeHTML(loan.issue)}</td><td>${escapeHTML(loan.due)}</td><td
class="${fine(loan) ? "fine" : ""}">$${fine(loan)}</td><td>${loan.returned ?
"Returned" : `<button class="button secondary small return-book"
data-id="${loan.id}">Mark returned</button>`}</td></tr>`).join("")}</tbody></table>` :
'<div class="empty">No books have been issued.</div>';
      document.querySelectorAll(".return-book").forEach(button =>
button.addEventListener("click", () => {
        const loan = db.loans.find(item => item.id === Number(button.dataset.id));
        if (loan) { loan.returned = true; loan.returnedOn = new
Date().toISOString().slice(0,10); save(); renderAdmin(); }
      }));
      $("#admin-materials").innerHTML = db.materials.length ?
`<table><thead><tr><th>Material</th><th>Year</th><th>Semester</th><th>Actions</th></tr
></thead><tbody>${db.materials.map(item =>
`<tr><td>${escapeHTML(item.title)}</td><td>${escapeHTML(item.year)}</td><td>${escapeHT
ML(item.semester)}</td><td><button class="button secondary small edit-material" data-
id="${item.id}">Edit</button> <button class="button danger small delete-material"
data-id="${item.id}">Delete</button></td></tr>`).join("")}</tbody></table>` : '<div
class="empty">No study materials added.</div>';
      document.querySelectorAll(".edit-material").forEach(button =>
button.addEventListener("click", () => {
        const material = db.materials.find(item => item.id ===
Number(button.dataset.id));
        if (!material) return;
        const form = $("#material-form");
        form.elements.materialId.value = material.id;
        form.elements.title.value = material.title;
        form.elements.year.value = material.year;
        form.elements.semester.value = material.semester;
        form.querySelector('[type="submit"]').textContent = "Update material";
        $("#cancel-material-edit").classList.remove("hidden");
        form.scrollIntoView({ behavior: "smooth", block: "center" });
      }));
      document.querySelectorAll(".delete-material").forEach(button =>
button.addEventListener("click", () => {
        db.materials = db.materials.filter(item => item.id !==
Number(button.dataset.id));
        save(); renderAdmin(); renderMaterials();
      }));
    }
    function updateAuth() {
      const loggedIn = !!currentUser;
      $("#login-open").classList.toggle("hidden", loggedIn);
      $("#logout").classList.toggle("hidden", !loggedIn);
      $("#user-label").textContent = loggedIn ? currentUser.name : "";
      $("#dashboard-nav").classList.toggle("hidden", !loggedIn || currentUser.role !==
"user");
```

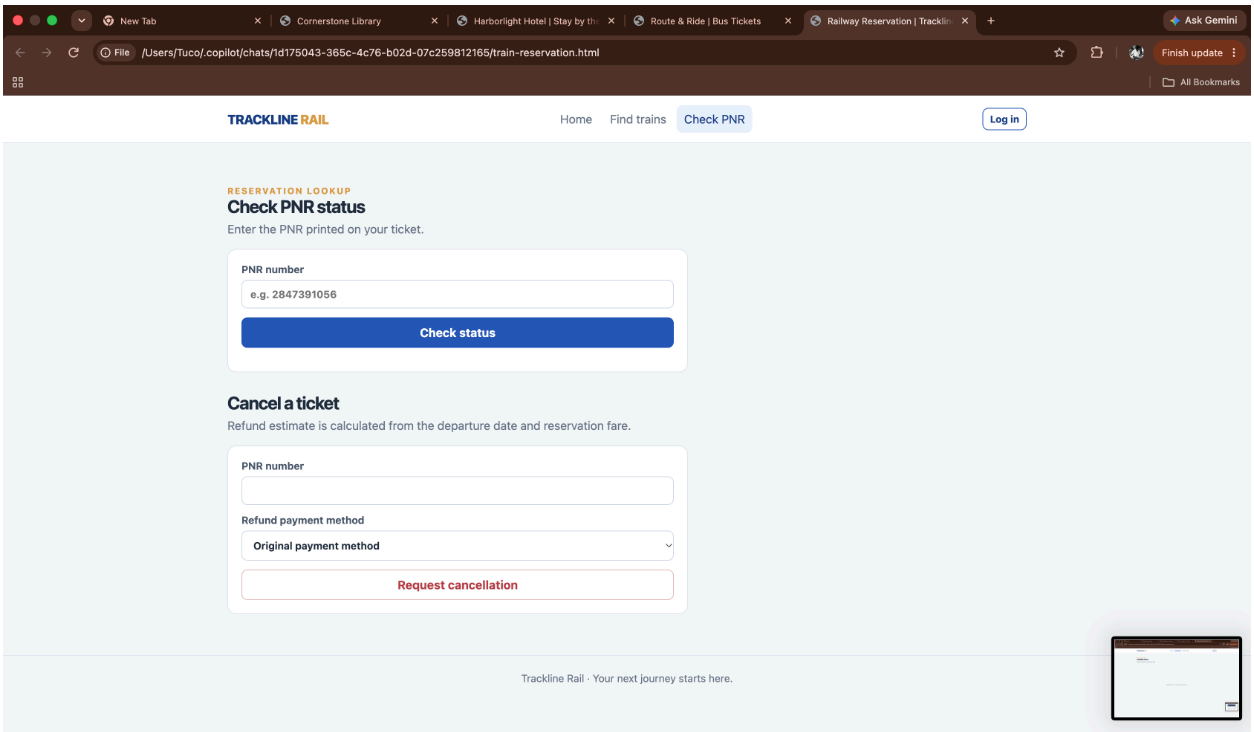
```
    $("#admin-nav").classList.toggle("hidden", !loggedIn || currentUser.role !== "admin");
  }
  document.querySelectorAll("nav [data-page]").forEach(button => button.addEventListener("click", () => go(button.dataset.page)));
  document.querySelectorAll("[data-go]").forEach(button => button.addEventListener("click", () => go(button.dataset.go)));
  $("#book-search").addEventListener("input", renderBooks);
  $("#material-year").addEventListener("change", renderMaterials);
  $("#material-semester").addEventListener("change", renderMaterials);
  $("#login-open").addEventListener("click", () => $("#auth-modal").classList.remove("hidden"));
  $("#modal-close").addEventListener("click", () => $("#auth-modal").classList.add("hidden"));
  $("#auth-modal").addEventListener("click", event => { if (event.target.id === "auth-modal") $("#auth-modal").classList.add("hidden"); });
  $("#show-login").addEventListener("click", () => { $("#login-form").classList.remove("hidden"); $("#register-form").classList.add("hidden"); $("#auth-title").textContent = "Log in"; });
  $("#show-register").addEventListener("click", () => { $("#register-form").classList.remove("hidden"); $("#login-form").classList.add("hidden"); $("#auth-title").textContent = "Create account"; });
  $("#login-form").addEventListener("submit", event => {
    event.preventDefault();
    const data = new FormData(event.currentTarget), email = String(data.get("email")).trim().toLowerCase();
    const user = db.users.find(item => item.email === email && item.password === data.get("password"));
    if (!user) return notice("Email or password is incorrect.", true);
    currentUser = user; updateAuth(); $("#auth-modal").classList.add("hidden"); event.currentTarget.reset();
    go(user.role === "admin" ? "admin" : "dashboard"); notice(`Signed in as ${user.name}.`);
  });
  $("#register-form").addEventListener("submit", event => {
    event.preventDefault();
    const data = new FormData(event.currentTarget), email = String(data.get("email")).trim().toLowerCase();
    if (db.users.some(user => user.email === email)) return notice("An account with this email already exists.", true);
    const user = { name: String(data.get("name")).trim(), email, password: String(data.get("password")), role: "user" };
    db.users.push(user); save(); currentUser = user; updateAuth(); $("#auth-modal").classList.add("hidden"); event.currentTarget.reset();
    go("dashboard"); notice("Your account is ready.");
  });
```

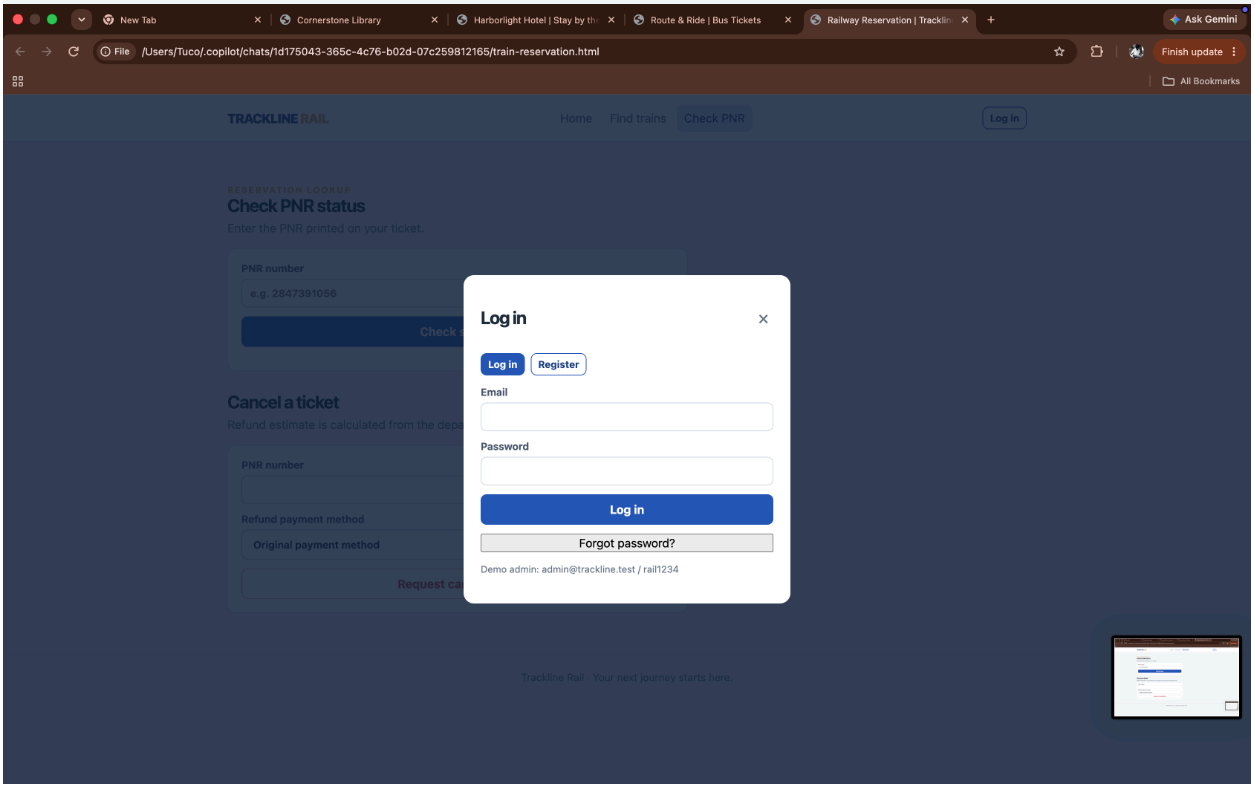
```
    $("#logout").addEventListener("click", () => { currentUser = null; updateAuth();
go("home"); notice("You have logged out."); });
    $("#book-form").addEventListener("submit", event => {
      event.preventDefault(); const data = new FormData(event.currentTarget);
      const id = Number(data.get("bookId"));
      const book = db.books.find(item => item.id === id);
      if (book) {
        const borrowed = book.copies - available(book);
        if (Number(data.get("copies")) < borrowed) return notice(`Copies cannot be
lower than the ${borrowed} currently issued.`, true);
        Object.assign(book, { title: String(data.get("title")).trim(), author:
String(data.get("author")).trim(), subject: String(data.get("subject")).trim(),
copies: Number(data.get("copies")) });
      } else {
        db.books.push({ id: Date.now(), title: String(data.get("title")).trim(),
author: String(data.get("author")).trim(), subject:
String(data.get("subject")).trim(), copies: Number(data.get("copies")) });
      }
      save(); event.currentTarget.reset(); event.currentTarget.elements.copies.min =
1; event.currentTarget.querySelector('[type="submit"]').textContent = "Add book";
$("#cancel-book-edit").classList.add("hidden"); renderAdmin(); renderBooks();
notice(book ? "Book updated." : "Book added to inventory.");
    });
    $("#cancel-book-edit").addEventListener("click", () => {
      const form = $("#book-form"); form.reset(); form.elements.copies.min = 1;
form.querySelector('[type="submit"]').textContent = "Add book";
$("#cancel-book-edit").classList.add("hidden");
    });
    $("#issue-form").addEventListener("submit", event => {
      event.preventDefault(); const data = new FormData(event.currentTarget), bookId =
Number(data.get("book"));
      const book = db.books.find(item => item.id === bookId);
      if (!book || available(book) < 1) return notice("No copies of this book are
currently available.", true);
      db.loans.push({ id: Date.now(), email: String(data.get("user")), bookId, issue:
new Date().toISOString().slice(0,10), due: String(data.get("due")), returned: false
});
      save(); renderAdmin(); notice("Book issued successfully.");
    });
    $("#material-form").addEventListener("submit", event => {
      event.preventDefault();
      const data = new FormData(event.currentTarget), file = data.get("file");
      const id = Number(data.get("materialId"));
      const material = db.materials.find(item => item.id === id);
      const finish = (fileData, filename) => {
        const values = { title: String(data.get("title")).trim(), year:
String(data.get("year")), semester: String(data.get("semester")) };
```

```
        if (material) Object.assign(material, values, fileData === null ? {} : { data:
fileData, filename });
        else db.materials.push({ id: Date.now(), ...values, filename, data: fileData
});
        save(); event.currentTarget.reset();
event.currentTarget.querySelector('[type="submit"]').textContent = "Add material";
$("#cancel-material-edit").classList.add("hidden"); renderAdmin(); renderMaterials();
notice(material ? "Study material updated." : "Study material added.");
      };
      if (file instanceof File && file.size) {
        const reader = new FileReader();
        reader.onerror = () => notice("Could not read the selected material file.",
true);
        reader.onload = () => finish(reader.result, file.name);
        reader.readAsDataURL(file);
      } else if (material) finish(null, material.filename);
      else notice("Choose a file to add the study material.", true);
    });
    $("#cancel-material-edit").addEventListener("click", () => {
      const form = $("#material-form"); form.reset();
form.querySelector('[type="submit"]').textContent = "Add material"; $("#cancel-
material-edit").classList.add("hidden");
    });
    $("#contact-form").addEventListener("submit", event => {
      event.preventDefault(); event.currentTarget.reset();
      notice("Thanks for your message. In this demo, connect the form to an email
service to deliver it.");
    });
    $("#featured-books").innerHTML = db.books.slice(0,3).map(bookCard).join("");
    renderBooks(); renderMaterials(); updateAuth();
  </script>
</body>
</html>
```

OUTPUT:


New Tab
Cornerstone Library
Harborlight Hotel | Stay by the
Route & Ride | Bus Tickets
Railway Reservation | Trackline
Ask Gemini
File /Users/Tuco/.copilot/chats/1d175043-365c-4c76-b02d-07c259812165
To exit full screen, press and hold esc
Finish update
All Bookmarks
TRACKLINE RAIL
Home
Find trains
Check PNR
Log in
YOUR JOURNEY, IN GOOD HANDS
See more.
Go by rail.
Find your train, choose your class, and reserve your seat with a few simple steps.
From station
Origin station
To station
Destination station
Journey date
23/10/2026
Search trains
EXPLORE THE NETWORK
Popular trains
Find a service and check seats by class.
Coastal zone
Coastal Express
Central → Riverside
TX204 · Departs 09:15 · 2026-10-23
Standard · $28
24 seats
Northern zone
North Star
Central → Northgate
TX318 · Departs 11:40 · 2026-10-23
Standard · $34
20 seats
Lakes zone
Lake Line
Riverside → Lakeside
TX507 · Departs 15:05 · 2026-10-23
Standard · $22
18 seats
New Tab
Cornerstone Library
Harborlight Hotel | Stay by the
Route & Ride | Bus Tickets
Railway Reservation | Trackline
Ask Gemini
File /Users/Tuco/.copilot/chats/1d175043-365c-4c76-b02d-07c259812165/train-reservation.html
Finish update
All Bookmarks
TRACKLINE RAIL
Home
Find trains
Check PNR
Log in
PLAN YOUR TRIP
Available trains
Search by stations and travel date.
Trackline Rail · Your next journey starts here.


New Tab
Cornerstone Library
Harborlight Hotel | Stay by the
Route & Ride | Bus Tickets
Railway Reservation | Trackline
Ask Gemini
File /Users/Tuco/.copilot/chats/1d175043-365c-4c76-b02d-07c259812165/train-reservation.html
Finish update
All Bookmarks
TRACKLINE RAIL
Home
Find trains
Check PNR
Log in
RESERVATION LOOKUP
Check PNR status
Enter the PNR printed on your ticket.
PNR number
e.g. 2847391056
Check status
Cancel a ticket
Refund estimate is calculated from the departure date and reservation fare.
PNR number
Refund payment method
Original payment method
Request cancellation
Trackline Rail · Your next journey starts here.


New Tab
Cornerstone Library
Harborlight Hotel | Stay by the
Route & Ride | Bus Tickets
Railway Reservation | Trackline
Ask Gemini
File /Users/Tuco/.copilot/chats/1d175043-365c-4c76-b02d-07c259812165/train-reservation.html
Finish update
All Bookmarks
TRACKLINE RAIL
Home
Find trains
Check PNR
Log in
RESERVATION LOOKUP
Check PNR status
Enter the PNR printed on your ticket.
PNR number
e.g. 2847391056
Cancel a ticket
Refund estimate is calculated from the depa
PNR number
Refund payment method
Original payment method
Request ca
Log in
Log in
Register
Email
Password
Log in
Forgot password?
Demo admin: admin@trackline.test / rail1234
Trackline Rail · Your next journey starts here.

RESULT : The Online Train Reservation System was developed to simplify railway ticket booking and reservation management. Passengers can log in, enter journey details, search for available trains, check seat availability, select travel classes, and book tickets through online or offline payment modes. After successful payment, the system generates tickets and PNR numbers, enabling passengers to check their reservation status, including Confirmed, RAC, and Waiting List. Passengers can also cancel tickets using their PNR numbers and receive refunds according to the applicable refund rules. Administrators can manage train details, schedules, fares, seat availability, payment records, cancellations, and refunds. Thus, the system improves reservation accuracy, reduces manual effort, and provides a user-friendly platform for passengers and railway administrators.